Game Success Predictor

Ronny Kicho || 30.10-2026

1. Problemområdet

Omfang

Spillindustrien består av et stort antall spill som lanseres hvert år, men ikke alle spill blir sett. For mange spillutviklere kan det være vanskelig å vurdere hvor godt et nytt spill vil gjøre det på markedet før det lanseres. Dermed blir målet med dette prosjektet å utvikle en maskinlæringsmodell som kan predikere om et spill vil få mye oppmerksomhet eller popularitet på Steam. Modellen skal trenes på et datasett med informasjon om over 125 000 eksisterende Steam-spill. Informasjonen er blant annet pris, sjanger, lanseringsperiode, støttede språk osv.

I dette prosjektet definerer vi foreløpig et populært spill som et spill med minst 100 000 estimerte eiere. Modellen skal dermed løse et binært klassifikasjonsproblem, der resultatet enten er «populært» eller «ikke populært». Maskinlæring er egnet til denne oppgaven fordi det kan være vanskelig å identifisere sammenhenger mellom et spills egenskaper og popularitet manuelt, spesielt med den store mengden spill som allerede finnes på Steam. Derfor kunne det vært gunstig å bruke ML til å finne mønstre i det store datasettet for å se om det statistisk sett er noe informasjon som direkte eller delvis fører til at spill kan bli mer populært.

Løsningen skal gjøres tilgjengelig gjennom en nettside der brukeren kan legge inn informasjon om et spill, for eksempel pris og sjanger. Nettsiden skal deretter bruke den trente maskinlæringsmodellen til å gi en prediksjon. Målgruppen er hovedsakelig spillutviklere og andre som er interessert i å undersøke hvilke egenskaper som kan ha sammenheng med et spills popularitet.

Metrikker

For å evaluere hvor godt maskinlæringsmodellen fungerer, bruker jeg fire forskjellige metrikker:

Accuracy viser hvor ofte modellen klarer å gjette riktig. Problemet er at datasettet er ganske ubalansert, siden bare rundt 7 % av spillene har over 100 000 estimerte eiere. Det betyr at modellen kan få hele 93 % accuracy bare ved å svare «ikke populært» på alle spillene, selv om den egentlig ikke klarer å finne noen populære spill.

Precision måler hvor stor andel av spillene modellen klassifiserer som populære, som faktisk er populære.

Recall måler hvor stor andel av alle de populære spillene modellen klarer å identifisere. Så om det er 100 populære spill, men bare klarer å identifisere 50 av de, har den en recall på 50%.

F1-score kombinerer precision og recall og vil derfor være en viktig metrikk for å vurdere modellens ytelse.

De tre siste metrikkene vil bli lagt spesielt mye vekt på på grunn av at datasettet er så ustabilt. Siden bare rundt 7% av spillene har over 100 000 estimerte eiere, hvis modellen alltid svarer "ikke populært", vil den få en accuracy score på 93%.

2. Data

Datasett

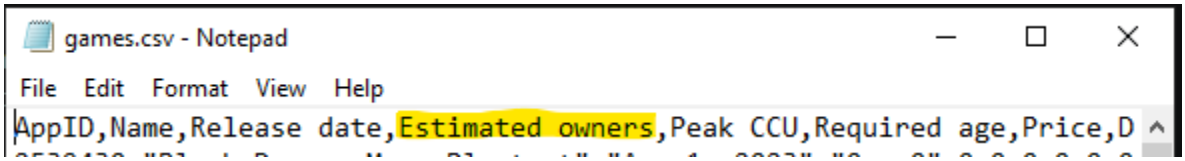
Jeg bruker et datasett med informasjon om 125 855 spill fra Steam. Datasettet inneholder 40 kolonner med forskjellige egenskaper ved spillene, blant annet pris, sjanger, lanseringsdato, støttede språk og estimert antall eiere.

Før jeg trener modellen må jeg rense dataen. Jeg fjerner spill som har en lanseringsdato frem i tid. Etter filtreringen står vi igjen med 125 854 spill. Det var altså bare ett spill i datasettet som ikke hadde blitt lansert ennå.

```
Antall spill før rensing: 125855
Antall spill etter rensing: 125854
Antall spill fjernet: 1
```

Figur 2.1: Rensing av data

Deretter bruker vi kolonnen "Estimated owners" til å lage verdien modellen skal predikere.



Figur 2.2: Datasett egenskaper (Estimated owners)

Problemet med datasettet jeg bruker er at det ikke gir nøyaktige verdier på hvor mange eiere et spill har, det gir verdier i intervaller. For eksempel kan et spill ha 100 000 - 200 000 "estimated owners". Vi har da valgt å bruke det laveste tallet i intervallet av de to tallene for å avgjøre om spillet er populært eller ikke. Så hvis et spill iallfall har 100 000 estimerte eiere, regner vi det som populært. Populære spill får da verdien 1, mens upopulære får verdien 0.

Valg av egenskaper

Vi bruker disse egenskapene som input til modellen:

- Pris
- Windows-støtte
- Mac-støtte

- Linux-støtte
- Sjanger

Jeg valgte disse egenskapene fordi de er vanligvis kjent før spillet blir lagt ut. Alle de 3 operativsystemene får enten en verdi på 1 eller 0, hvor 1 betyr at de støttes og 0 betyr at de ikke støttes. Siden et spill kan ha flere sjangere, gjør vi sjangerkolonnen til flere egne kolonner hvor hver sjanger får verdien 1 hvis spillet tilhører sjangeren og 0 hvis ikke.



	RPG	Action	Strategy
Spill A	0	1	1
Spill B	0	0	1
Spill C	1	1	1

Figur 2.3: Sjangertabell

Med egenskapene som er nevnt i tillegg til alle de forskjellige sjangerne har vi da totalt 37 egenskaper som brukes til å trene modellen.

Fordeling av data

Jeg deler dataen i treningsdata og testdata. 80% av spillene brukes til trening, mens resten blir brukt til testing. Jeg bruker train_test_split fra scikit-learn og jeg bruker også stratify for å sørge for at både treningsdataene og testdataene inneholder omtrent 7 % populære spill og 93 % ikke-populære spill, slik som i det opprinnelige datasettet.

Begrensinger med datasettet

En begrensning med datasettet er at vi ikke vet nøyaktig hvor mange eiere hvert spill har, siden vi bare har estimerte intervaller. Dette kan påvirke hvor riktig modellen blir trent.

Et annet problem er at eldre spill har hatt mye lengre tid på å få flere eiere enn nyere spill. Et spill som ble lansert for 10 år siden, har for eksempel hatt mye mer tid på å nå 100 000 eiere enn et spill som ble lansert for noen måneder siden.

Vi bruker heller ikke alle egenskapene som finnes i datasettet. For eksempel kan markedsføring, anmeldelser og hvor kjent utvikleren er, ha mye å si for hvor populært et spill blir. Dette er informasjon modellen min ikke tar hensyn til og mye av det er rett og slett bare ting man ikke kan forutsi før man faktisk har lansert spillet sitt.

3. Modellering

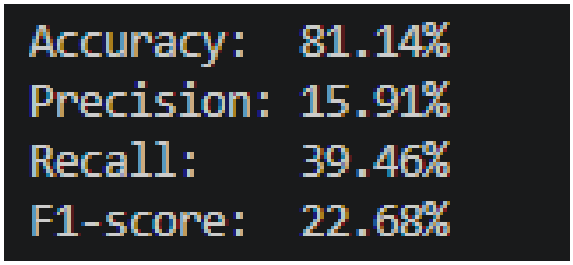
Valg av modell

Jeg har valgt å bruke Random Forest til å trene modellen. Random Forest er en maskinlæringsalgoritme som består av flere beslutningstrær. Hvert beslutningstre prøver å klassifisere spillet basert på egenskapene det har, og Random Forest kombinerer resultatene fra trærne for å komme frem til en prediksjon.

Jeg valgte Random Forest fordi den kan brukes til klassifisering, som passer bra til prosjektet mitt siden modellen bare skal velge mellom to mulige resultater: populært eller ikke populært. Jeg bruker RandomForestClassifier fra scikit-learn, og har satt antall beslutningstrær til 100. Jeg bruker også class_weight="balanced" fordi datasettet er ganske ubalansert. Som nevnt tidligere er bare rundt 7 % av spillene populære. Ved å bruke balanced får de populære spillene større vekt under treningen, slik at modellen ikke bare lærer seg å svare «ikke populært» på nesten alle spill.

Resultater

Etter at modellen var ferdig trent fikk jeg disse resultatene:

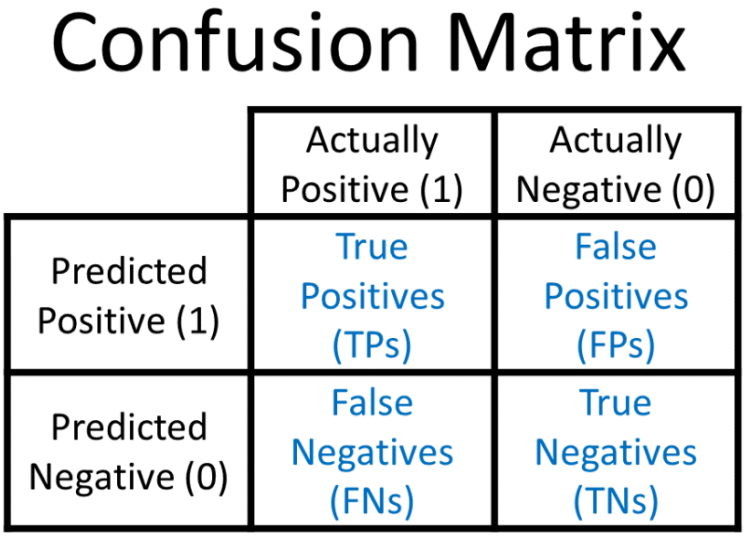


Figur 3.1: Resultater

Modellen fikk en accuracy score på 81,14% som i seg selv egentlig ikke har så mye å si med tanke på det jeg nevnte tidligere om at datasettet er ustabilt. Så accuracy alene er ikke nok til å vurdere hvor bra modellen fungerer, og det ser vi også når vi går gjennom resten av resultatene. Precision fikk bare 15,91%, recall ble 39,46% og kombinasjonen av de to metrikkene, altså F1-score, ble 22,68%. Så modellen har altså fortsatt en del problemer som absolutt kan forbedres.

Confusion matrix

For å se litt nærmere på hvilke feil modellen gjør, bruker jeg en confusion matrix, den viser hvor mange spill modellen har klassifisert riktig eller feil gjennom TP(True Positives), FP(False Positives), TN(True Negatives) og FN(False Negatives), slik som figur 3.2 viser.



Figur 3.2: Confusion matrix

Tabellen i figur 3.2 blir brukt som utgangspunkt til å printe ut confusion matrixen.

```
Confusion matrix:
[[  696  3678]
 [ 1068 19729]]
```

Figur 3.3: Printet confusion matrix

Figur 3.3 viser at modellen klarte å identifisere 696 populære spill riktig, samtidig var det 1068 populære spill som modellen ikke klarte å identifisere. Et annet problem er at modellen klassifiserte 3678 upopulære spill som populære. Det forklarer hvorfor Precision ble så lav, modellen prøver å finne populære spill, men ender også opp med å klassifisere mange upopulære spill som populære.

Baseline

For å ha noe å sammenligne modellen med, kan vi bruke en veldig enkel løsning som alltid svarer «ikke populært». Siden omtrent 93 % av spillene i datasettet er upopulære, ville denne løsningen fått rundt 93 % accuracy. Det høres kanskje bedre ut enn modellen min, som fikk 81 %, men problemet er at denne løsningen aldri ville klart å identifisere et populært spill. Den ville derfor fått 0 % recall og en F1-score på 0 % for populære spill.Random Forest-modellen min har lavere accuracy, men den klarer iallfall å identifisere 39 % av de populære spillene. Derfor mener jeg at den er mer nyttig enn en modell som bare svarer «ikke populært» hele tiden.

Vurdering av resultatene

Resultatene viser at modellen fungerer, men at den ikke er spesielt nøyaktig når den skal identifisere populære spill. En mulig grunn til dette er at jeg bare bruker pris, sjanger og støttede operativsystemer som input. Det er mange andre ting som påvirker om et spill blir populært, som for eksempel markedsføring, kvaliteten på spillet og hvor kjent utvikleren er.

Det kan også være vanskelig å predikere popularitet bare ved å se på egenskapene til et spill. To spill kan ha samme pris og sjanger, men likevel få veldig forskjellige antall spillere. Det er såpass mange faktorer som går inn og data som ikke er tilgjengelig som kunne forbedret modellen betydelig. Som for eksempel om jeg hadde informasjon på hvor mange timers arbeid som gikk inn i utviklingen av hvert spill, eller hvilken game engine som ble brukt etc. Alt dette er med på å minke F1-scoren.

Selv om modellen ikke fikk veldig gode resultater, har jeg klart å trene en modell som kan finne noen av de populære spillene basert på informasjonen den får. Den klarer å finne noen mønstre og klarer noen ganger.

4. Deployment

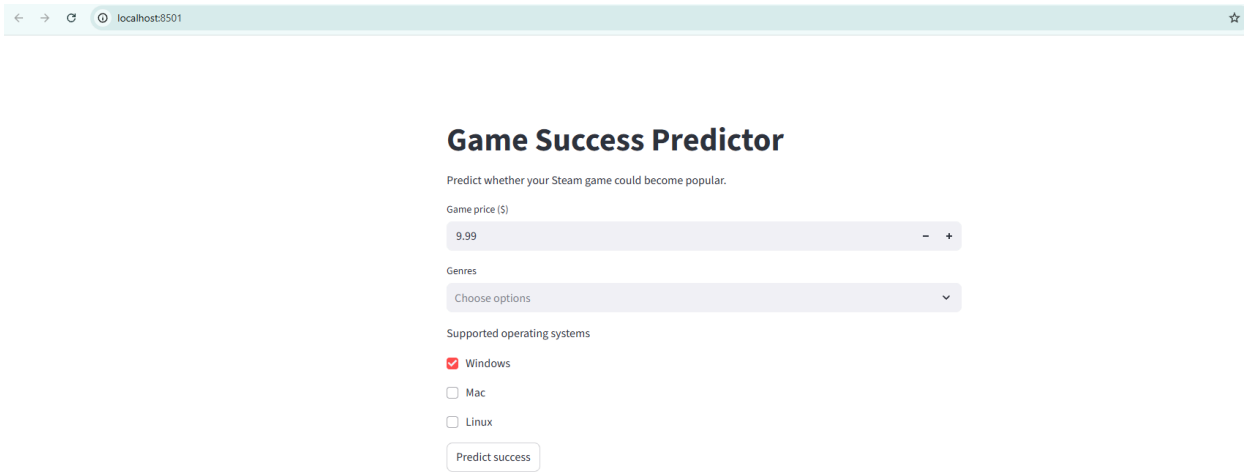
Nettsiden

For å gjøre det litt enklere for meg selv òg lett for å bruke modellen, har jeg laget en nettside med Streamlit. Streamlit er et python-bibliotek som gjør det lett å lage en nettside uten å måtte bruke html, css eller Javascript.

Som nevnt tidligere har vi fem egenskaper vi vil bruke som input, så på nettsiden kan brukeren legge inn følgende informasjon om spillet de vil predikere:

- pris
- sjanger
- Windows-Støtte
- Mac-Støtte
- Linux-Støtte

Brukeren kan velge flere sjangere samtidig siden et spill kan tilhøre flere forskjellige sjangere. Når de har fylt inn informasjonen, kan de trykke på knappen “Predict success”. Nettsiden bruker da modellen til å predikere om spillet vil være populært eller ikke.



Figur 4.1: Nettside

Hvordan nettsiden fungerer

Prosjektet mitt består hovedsakelig av to python-filer, train.py og app.py. train-scriptet brukes til å lese inn datasettet, rense dataen og trene Random Forest-modellen. Etter modellen er ferdig trent, lagrer jeg den i en fil som heter model.pkl ved hjelp av joblib. Dette gjør at jeg slipper å trene modellen på nytt hver gang jeg skal bruke nettsiden.

app.py brukes til å kjøre selve nettsiden. Denne filen laster inn den ferdigtrente modellen fra model.pkl. Når brukeren fyller inn informasjon om spillet, blir denne informasjonen gjort om til de samme egenskapene som modellen ble trent med. Modellen bruker deretter informasjonen til å gjøre en prediksjon.

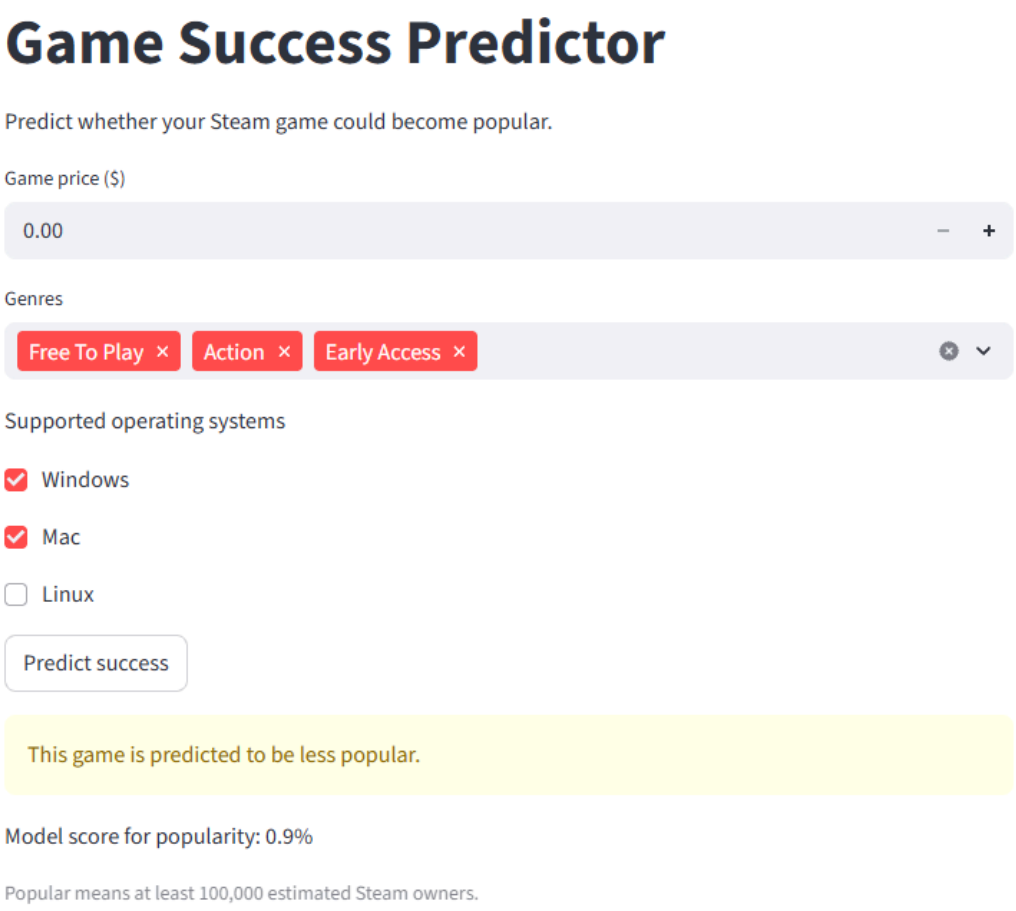
Hvis modellen klassifiserer spillet som populært, får brukeren beskjed om dette. Hvis modellen klassifiserer det som ikke populært, får brukeren en annen beskjed. Nettsiden viser også en score for hvor populært modellen vurderer at spillet kan bli. Denne scoren er basert på modellens beregninger og er ikke en garanti for at spillet faktisk blir populært.

Testing av nettsiden

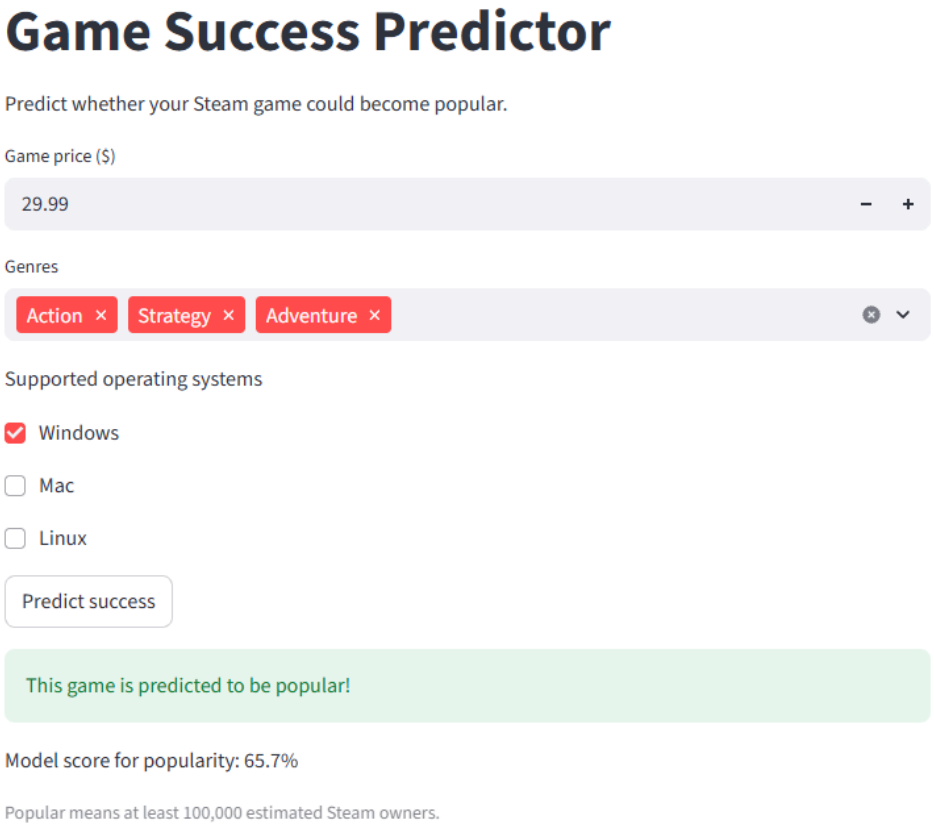
Jeg har kjørt nettsiden lokalt ved hjelp av Streamlit. Nettsiden åpnes i nettleser, og brukeren kan fylle inn informasjon om et spill.

For å teste nettsiden har jeg prøvd forskjellige kombinasjoner av pris, sjanger og operativsystemer:

Test 1:



Test 2:



den første testen prøvde jeg et gratis spill med sjangerne Free To Play, Action og Early Access. Modellen klassifiserte dette spillet som ikke populært og ga en score på 0,9 %.I den andre testen prøvde jeg et spill som kostet 29,99 dollar, med sjangerne Action, Strategy og Adventure. Denne gangen klassifiserte modellen spillet som populært og ga en score på 65,7 %.

Begge testene viser at nettsiden klarer å ta imot informasjon fra brukeren og bruke modellen til å gjøre en prediksjon. Resultatene viser også at modellen kan gi forskjellige prediksjoner basert på egenskapene som blir valgt.

Videre utvikling

Selv om nettsiden og modellen fungerer, er det fortsatt flere ting som kunne blitt forbedret. For eksempel kunne jeg brukt flere egenskaper fra datasettet for å se om modellen klarer å identifisere populære spill bedre. Jeg kunne også prøvd andre maskinlæringsalgoritmer og sammenlignet resultatene. Som for eksempel å lage en algoritme hvor modellen ser på utgiveren, om dette er en indie developer eller ikke. Popularitet fra gamle spill med suksess kan absolutt påvirke nyere utgivelser.

En annen forbedring kunne vært å bruke nyere data og trene modellen på nytt etter hvert som flere spill blir lansert. Nettsiden kjører bare lokalt på datamaskinen min, men hvis jeg skulle gjort den tilgjengelig for andre, kunne jeg publisert den på internettet.

Dersom modellen skulle brukes over lengre tid, ville det også vært viktig å følge med på hvor gode prediksjonene er, og oppdatere modellen dersom resultatene blir dårligere.

5. Referanser

Datasett:

https://huggingface.co/datasets/FronkonGames/steam-games-dataset(FronkonGames) 15.10.2026

Python-biblioteker:

- pandas - brukt til å lese inn og behandle data. https://pandas.pydata.org/docs
- scikit-learn - brukt til å trene og teste modellen. https://scikit-learn.org/stable
- streamlit - brukt til å lage nettsiden. https://docs.streamlit.io
- joblib - brukt til å lagre og laste inn modellen. https://joblib.readthedocs.io/en/stable

Figurer:

- Figur 2.1 - skjermbilde (Ronny Kicho) 20.10.2026
- Figur 2.2 - skjermbilde (Ronny Kicho) 20.10.2026
- Figur 2.3: - figur laget i paint (Ronny Kicho) 21.10.2026
- Figur 3.1 - skjermbilde (Ronny Kicho) 22.10.2026
- Figur 3.2 - https://glassboxmedicine.com/2019/02/17/measuring-performance-the-confusion-matrix/ (Rachel Draelos) 22.10.2026
- Figur 3.3 - skjermbilde (Ronny Kicho) 22.10.2026
- Figur 4.1 - skjermbilde (Ronny Kicho) 27.10.2026